COURSE OUTLINE

Gennoor Tech
TRAIN. INNOVATE. BUILD.

Credential and Access Management Automation with Claude

From a new-joiner record to the right accounts, the right tickets and a full audit trail

3 days (8 hours per day) | Intermediate | Instructor-led, hands-on labs | Capstone project

About this course

Who should attend

Enterprise IT and network operations teams, identity and access administrators, ServiceNow process owners, load balancer and data centre network engineers, platform and DevOps engineers in India and worldwide.

Prerequisites

- Basic Python and REST API concepts
- Working knowledge of at least one load balancer or network controller (F5, NetScaler, Radware or Cisco ACI)
- Familiarity with ServiceNow request and ticket workflows
- Git basics; familiarity with an AI coding assistant is helpful, not required

What you will be able to do

- Model a new-joiner access request as a typed record and a versioned role-to-entitlement matrix that Claude plans against
- Set up Claude Code, the Claude Agent SDK and MCP servers so the agent can read ServiceNow and identity data as tools and return structured JSON access plans
- Raise and route ServiceNow requests and requested items per system through the Table API and Flow Designer on behalf of the joiner
- Provision least-privilege local accounts on F5 (iControl REST and tmsh), Citrix NetScaler (NITRO API), Radware Alteon / Cyber Controller, Cisco APIC and other load balancers (Anaya LB) through deterministic, dry-run-first adapters
- Codify GitHub team and repository access as Terraform in a pull request, planned by GitHub Actions and applied only after review

- Enforce guardrails, human approval, one-time credential delivery and a complete audit log, and run the same workflow in reverse for leavers

Tools and technologies

Claude Code and Agent SDK | Python | Pydantic | ServiceNow Table API and Flow Designer | F5 iControl REST and tmsh | Citrix NetScaler NITRO API | Radware Alteon / Cyber Controller | Cisco APIC REST API | GitHub | GitHub Actions | Terraform | Secrets and one-time delivery

Table of contents

DAY 1 3 modules

MODULE 1

The new-joiner access workflow and the automation baseline

- Walking the real flow: HR trigger, manager request, per-system tickets, local device accounts, repository access, hand-over to the joiner
- Which systems network operations teams actually touch: F5, NetScaler, Radware, Cisco ACI, ServiceNow, GitHub and Terraform, plus other load balancers (Anaya LB)
- The teaching pattern used in every build: Claude plans, drafts and explains; deterministic code executes; a human approves; everything is logged
- Mapping tribal knowledge into a role-to-entitlement matrix with owners, scopes and expiry

HANDS-ON LAB
Walk through a sample joiner record and the team's current runbook, then draft the entitlement matrix for three roles as a versioned YAML file.

DELIVERABLE
Documented workflow and entitlement matrix v1.

Gennoor Tech INNOVATE. BUILD. For more info: https://gennoor.com

MODULE 2

Setting up Claude Code, the Agent SDK and MCP for the access workflow

- Claude Code project layout, CLAUDE.md conventions and permission settings for a credential workflow
- Claude Agent SDK orchestrator: tool definitions for read_joiner, lookup_entitlements, draft_ticket and plan_account
- MCP servers that expose ServiceNow records and the HR or identity source as read-only tools
- Structured JSON outputs validated with Pydantic so every access plan is an object, never free text

HANDS-ON LAB
Stand up the project, connect the ServiceNow MCP server to a developer instance, and have the agent return a validated access plan for one joiner.

DELIVERABLE
Working agent skeleton with MCP tools and an access-plan schema.

MODULE 3

Joiner intake and entitlement resolution

- Typed joiner record: employee ID, team, role, location, start date, manager, requested systems and justification
- Resolving role to entitlements per system: F5 partition and role, NetScaler command policy, APIC security domain and role, GitHub teams
- Rejecting incomplete or out-of-matrix requests with an explicit reason instead of a guess
- Diffing the proposed plan against what the joiner already has

HANDS-ON LAB
Implement the intake validator and entitlement resolver with Claude Code, then feed it a malformed record and an out-of-matrix request and inspect the rejections.

DELIVERABLE
Validated joiner record and per-system entitlement plan.

DAY 2 4 modules

Gennoor Tech TRAINING INNOVATE BUILD. For more info: https://gennoor.com

MODULE 4

Raising tickets in the right channel with ServiceNow

- ServiceNow Table API: creating sc_request and one sc_req_item per system, setting assignment_group, requested_for and short_description
- Letting the agent draft the ticket description and justification from the plan while the Python client owns the call
- Flow Designer for manager approval, state transitions and the callback that unblocks provisioning
- Idempotency: never raise a duplicate ticket for the same joiner and system

HANDS-ON LAB

Raise the full set of tickets for one joiner against the developer instance, route each to its fulfilment group and watch Flow Designer approval gate the next step.

DELIVERABLE

Ticket orchestrator with per-channel routing and approval callback.

MODULE 5

Device accounts on F5 and Citrix NetScaler

- F5 iControl REST: POST /mgmt/tm/auth/user with partition access and role, and the equivalent tmsh create auth user command for review
- Citrix NetScaler NITRO API: /nitro/v1/config/systemuser and binding a systemcmdpolicy for least privilege
- Dry run first: the adapter prints the exact request body and tmsh line, then applies only after approval
- Verifying the account with a read-back and a test login, and recording the result

HANDS-ON LAB

Build the F5 and NetScaler adapters, run them in dry-run mode against lb-f5-01 and lb-ns-01 in the lab, then apply with approval and verify the accounts.

DELIVERABLE

F5 and NetScaler provisioning adapters with dry-run and verification.

Gemnoor Tech PRIVATE LIMITED
For more info- https://gemnoor.com

MODULE 6

Radware, Cisco ACI and other load balancers (Anaya LB)

- Radware Alteon and Cyber Controller: local user and role creation, and where the API stops and a ticketed manual step begins
- Cisco APIC REST API: creating aaaUser objects with aaaUserDomain and aaaUserRole for the right security domain
- An adapter interface so other load balancers (Anaya LB) plug in without changing the orchestrator
- Falling back to a ticket with exact instructions when a platform has no supported API path

HANDS-ON LAB

Implement the APIC adapter against the ACI sandbox and the Radware adapter against the lab controller, and prove the orchestrator raises a manual ticket for an unsupported platform.

DELIVERABLE

Multi-platform adapter set with graceful manual fallback.

MODULE 7

Repository and infrastructure access as code with GitHub and Terraform

- GitHub REST API for organisation invitations and team membership, and why access should live in Terraform instead
- Terraform github provider: github_team_membership and github_repository_collaborator, with terraform plan, apply and state as the record
- Claude Code opens the pull request with the plan output in the description; GitHub Actions runs plan on the PR and apply on merge
- Protecting the Terraform state and the GitHub token used by Actions

HANDS-ON LAB

Have Claude Code draft the Terraform change for the joiner, open the pull request, review the Actions plan output and merge to apply.

DELIVERABLE

Access-as-code pipeline from PR to applied state.

DAY 3 3 modules

GennoorTech INNOVATE. BUILD. For more info: https://gennoor.com

MODULE 8

Guardrails, approvals and audit for credential workflows

- Credentials are generated by code and delivered through a one-time channel; the model never sees a password
- Least privilege enforced by the matrix: the agent cannot request a role the matrix does not grant
- Human approval gates on every create, modify and delete, with the approver and decision recorded
- Append-only audit log of plan, approval, API call, response and verification, kept with the ticket

HANDS-ON LAB

Attempt to make the agent grant an administrator role outside the matrix, watch the guardrail refuse, and trace one complete joiner through the audit log.

DELIVERABLE

Guardrail policy, approval gate and audit log.

MODULE 9

Verification, hand-over and reversal for leavers and movers

- Post-provisioning verification: read-back on every platform, test authentication, ticket closure in ServiceNow
- Hand-over message to the joiner and manager drafted by Claude from the verified results
- Running the workflow in reverse: disable and remove accounts, revoke GitHub teams, record the leaver
- Periodic access review that compares live accounts with the matrix and raises tickets for drift

HANDS-ON LAB

Verify the joiner end to end, send the hand-over, then run the leaver workflow and prove every account and team membership is gone.

DELIVERABLE

Verification, hand-over and offboarding workflow.

GermoorTech
For more info - https://germoor.com

MODULE 10

Capstone: one joiner, every system, fully audited

- Joiner record in, tickets raised, accounts provisioned, repository access applied, hand-over sent, with no credential created without approval
- Run the leaver reversal on the same joiner and show the audit log closes every entry
- Operating the workflow after the course: matrix ownership, adapter versioning and periodic access review
- Acceptance criteria review, limitations statement, demo to the group

HANDS-ON LAB
Capstone build and demo.

DELIVERABLE
End-to-end access automation workflow and runbook.

Capstone

Capstone project

A repeatable workflow that takes a new-joiner record, resolves entitlements from a versioned matrix, raises the right ServiceNow tickets in the right channels, provisions least-privilege accounts on F5, NetScaler, Radware, Cisco ACI and other load balancers (Anaya LB), applies GitHub and Terraform access through a reviewed pull request, and leaves a complete audit log, with the same workflow running in reverse for leavers.

Assessment

Assessment area	Weight	Evidence
Working functionality	25%	Demonstrable end-to-end workflow in a repeatable lab
Security and safety controls	20%	Threat model, authorization boundaries, approval gates, safe failure
Testing and evaluation	20%	Automated tests, golden dataset, measured quality and known limitations
Engineering quality	15%	Clear interfaces, error handling, reproducible setup, maintainable code
Evidence and explainability	10%	Traceable findings, sources, assumptions, logs

Documentation and presentation	10%	Architecture diagram, README, runbook, demo, lessons learned

Engineering principles used throughout

- Model output is a proposal, not a fact: every finding and generated change is validated by deterministic checks and tests.
- Authorization lives outside the model: tool access is enforced by application code, scoped credentials and policy.
- Human approval for consequential actions: deploying a mitigation, changing production configuration, rotating credentials.
- Decisions, evidence, model and tool versions, approvals and outcomes are logged without recording secrets.
- A model adapter and a tool adapter layer keep every build provider-agnostic and enterprise-portable.
- Quality is measured: false positives, false negatives, reliability, latency, cost and safe failure, not just a demo.

© Gennoor Tech · Train. Innovate. Build.

For more info - https://gennoor.com